

프로그래밍 역량 강화 전문기관, 민코딩

Prime Factors



목차

1. Prime Factors 소개
2. Prime Factors 프로젝트 준비
3. Prime Factors TDD 개발

Prime Factors 소개

Prime Factors 소개

✓ 소인수 분해

- 양의 정수를 소수들의 곱으로 표현

✓ 예시 (수를 넣으면, 수인수분해 된 배열이 return 된다)

- $2 \rightarrow [2]$
- $3 \rightarrow [3]$
- $4 \rightarrow [2, 2]$
- $6 \rightarrow [2, 3]$
- $8 \rightarrow [2, 2, 2]$
- $9 \rightarrow [3, 3]$
- $12 \rightarrow [2, 2, 3]$
- $14 \rightarrow [2, 7]$

TDD 실습 규칙

✓ 한번에 전부 구현하지 않는다.

- 아주 사소한 것을 조금씩 점진적으로 구현한다.
(하드코딩 해도 된다.)

✓ 아주 간단한 테스트부터 시작한다.

- 그리고 Pass 되도록 구현한다.
이 코드는 아주 특수한 경우에만 동작하게 된다. (하드코딩 했으므로)

✓ 점차 범용적인 코드로 만든다.

- 테스트가 점점 구체화될수록,
코드는 아주 특수한 경우에만 동작하면서
점점 범용적인 코드로 변하게 된다.

테스트 코드도 리팩토링을 해야할까?

- ✓ **테스트 코드도 리팩토링을 하는 것이 좋다.**
 - Unit Test 코드는 관리되어야 하기 때문이다.

프로젝트, Repository 준비 + 실습을 위한 간단한 Git 복습

Prime Factors 프로젝트 준비

기본 코드 작성 & 창 배치하기

왼쪽 창 : Test 코드 파일 (prime-factors_test.cpp)

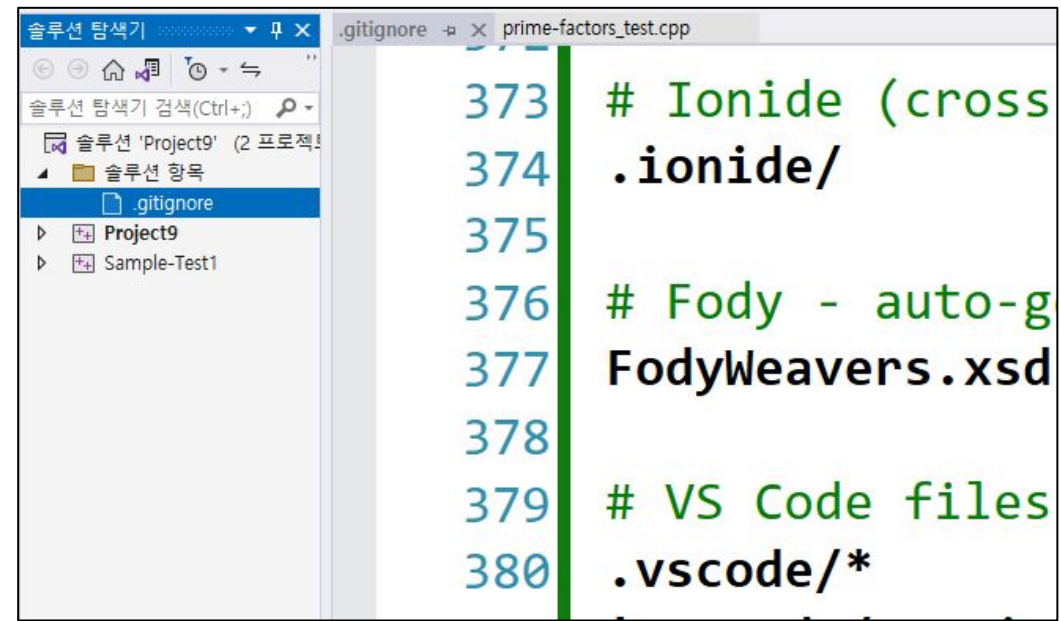
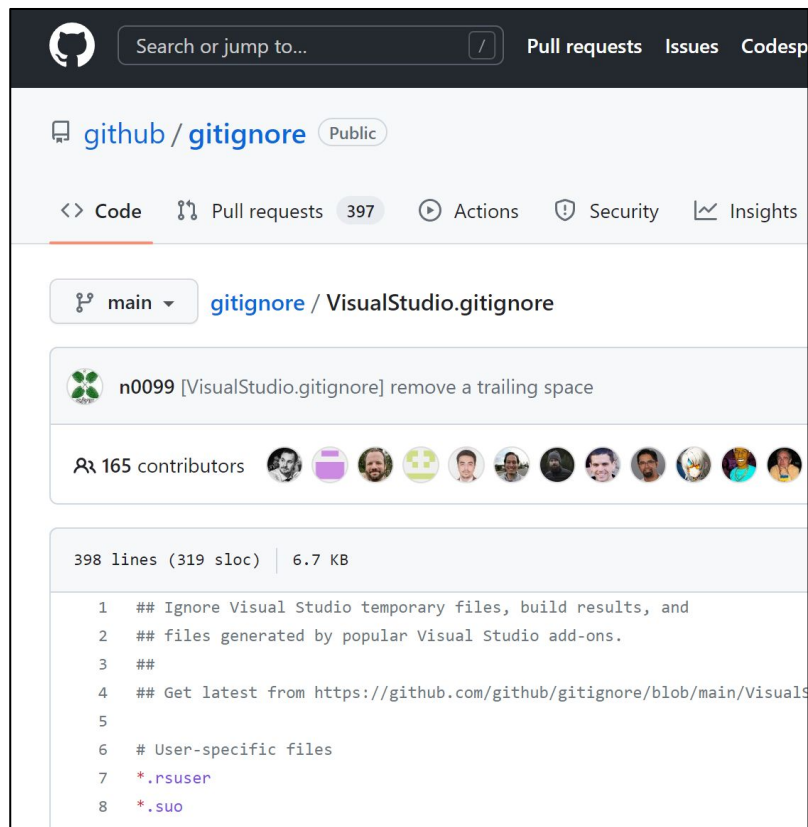
오른쪽 창 : 개발 소스코드 파일 (prime-factors.cpp)

```
Test
prime-factors_test.cpp
1 #include "pch.h"
2 #include "../Project9/prime-factors.cpp"
3
4 using namespace std;
5
6 TEST(PrimeFactors, PrimeTest) {
7     EXPECT_EQ(1, 1);
8 }

Production
prime-factors.cpp
1 class PrimeFactor {
2
3 };
```

.gitignore 추가

✓ 구글링 : github gitignore 검색 후, visual studio 의 .gitignore



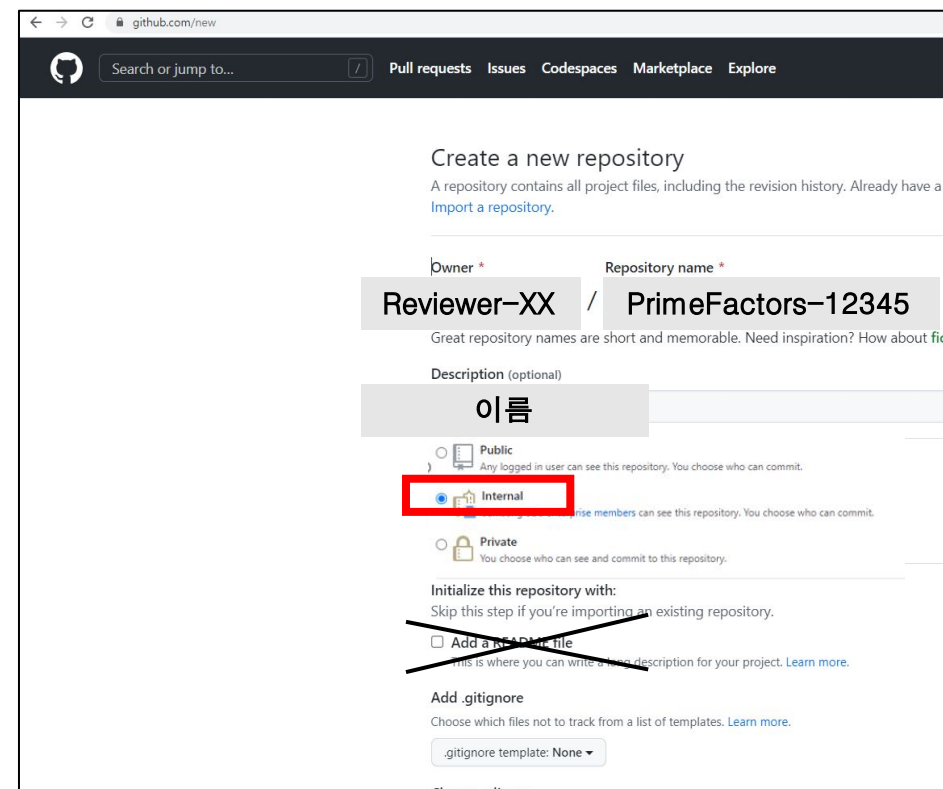
Remote Repository 생성

✓ Github Repository 생성하기

- Repo. name : PrimeFactors-번호
- Description : 이름
- Internal 선택
- README.md 파일 생성 안함

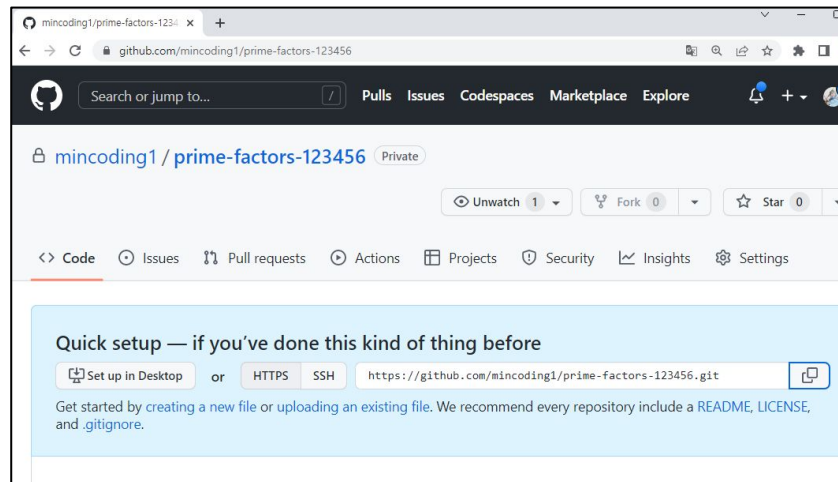
✓ 생성 후 설정

- Setting > Collaborator : 팀원 추가하기



현재까지 내용 Push

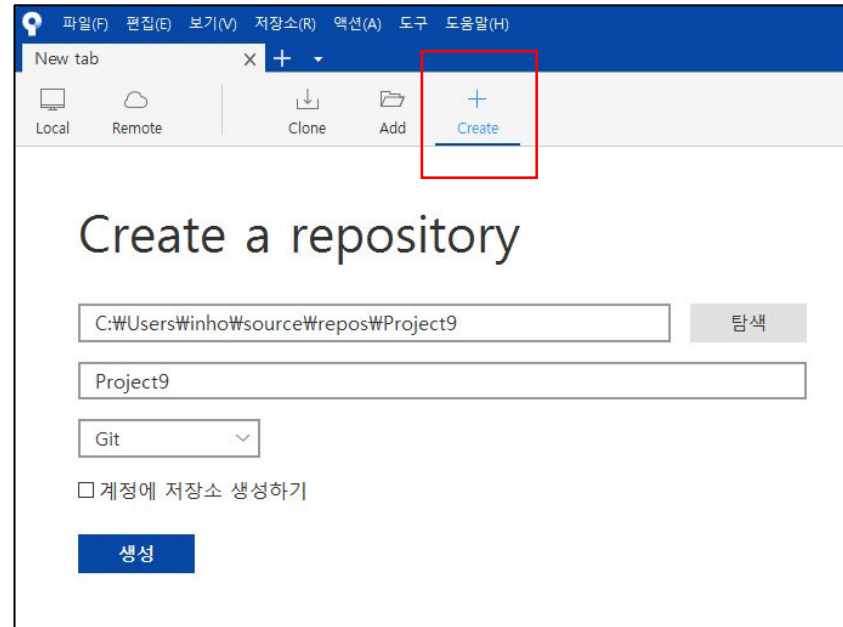
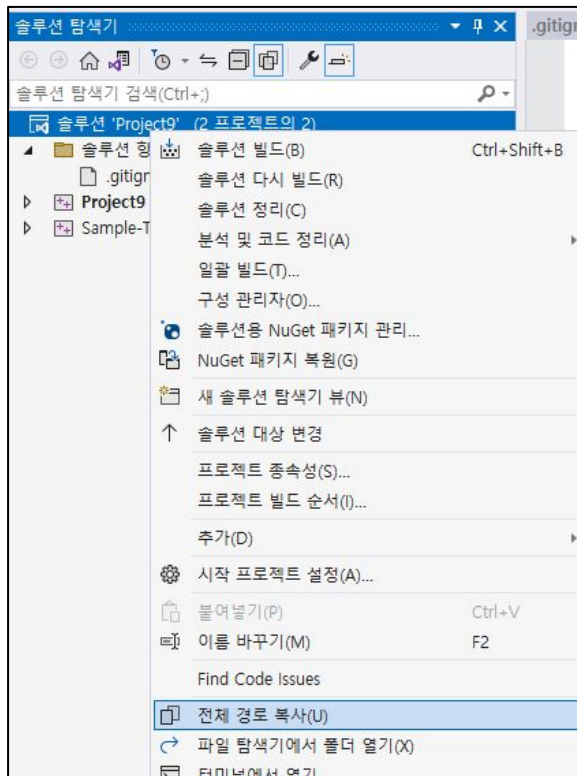
✓ Git GUI 도구를 이용하여 Push 진행



Github에서
Repo 생성 / Git 주소 복사하기

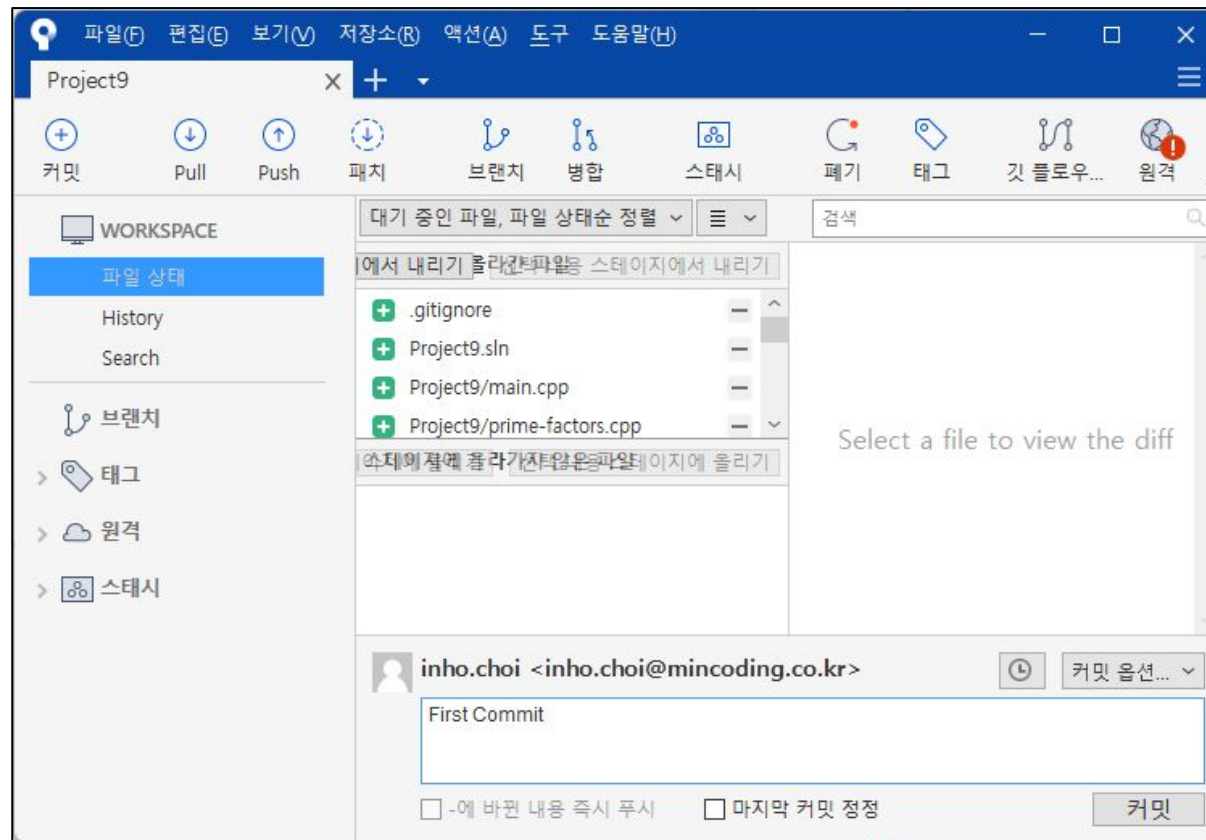
git init 수행

✓ Git 초기화 수행



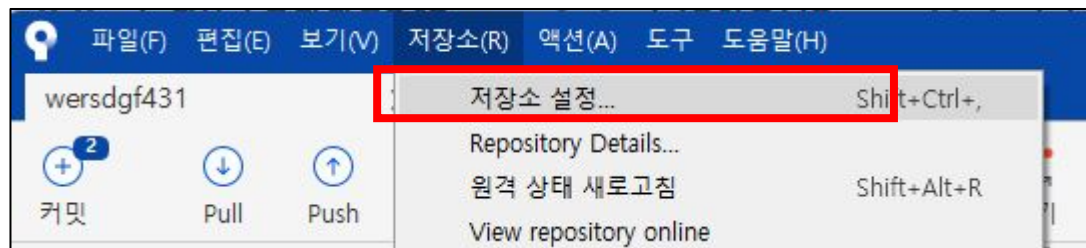
첫 Commit 진행

✓ Staging > Commit



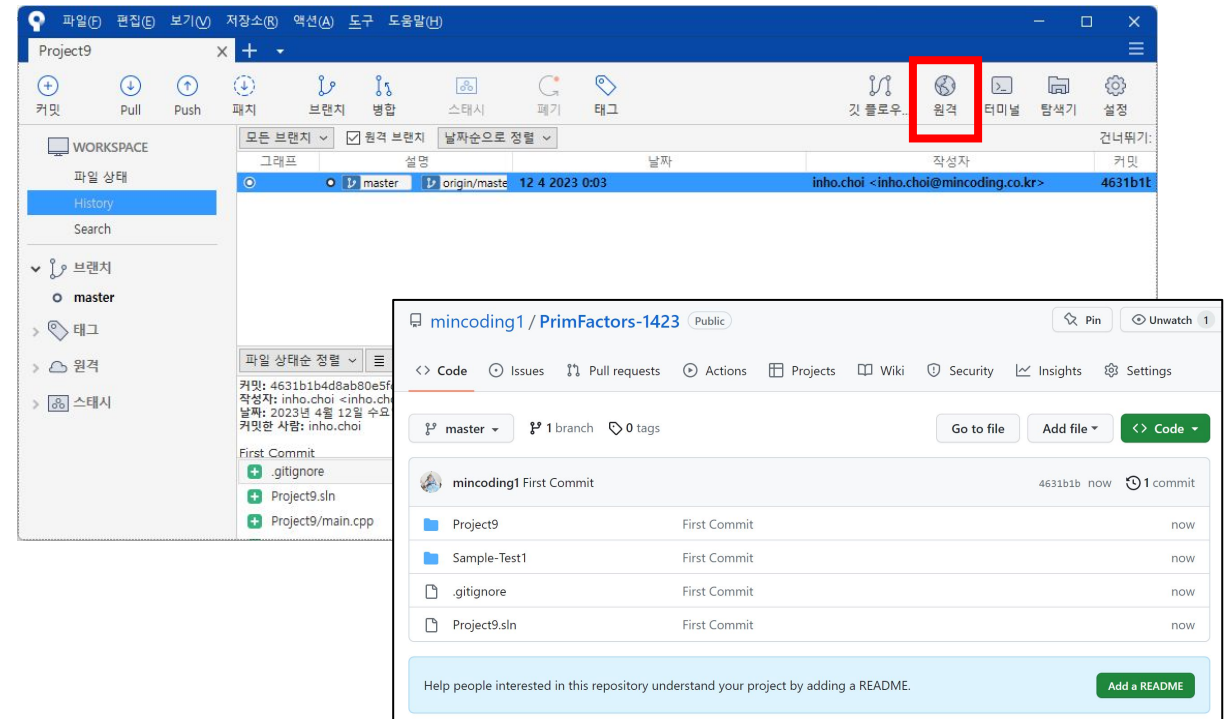
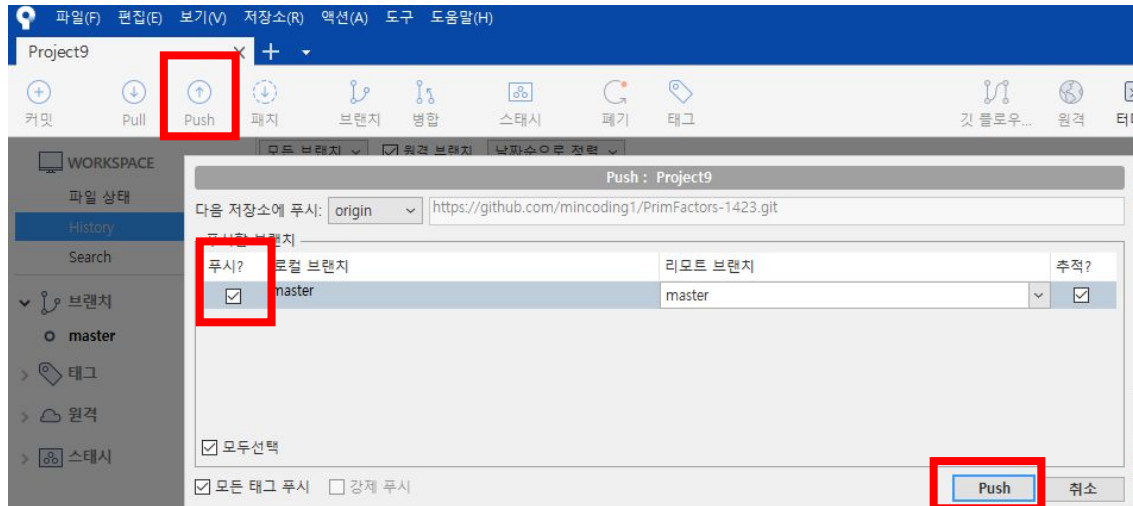
Remote 설정하기

✓ 생성한 Remote Repository 로 지정



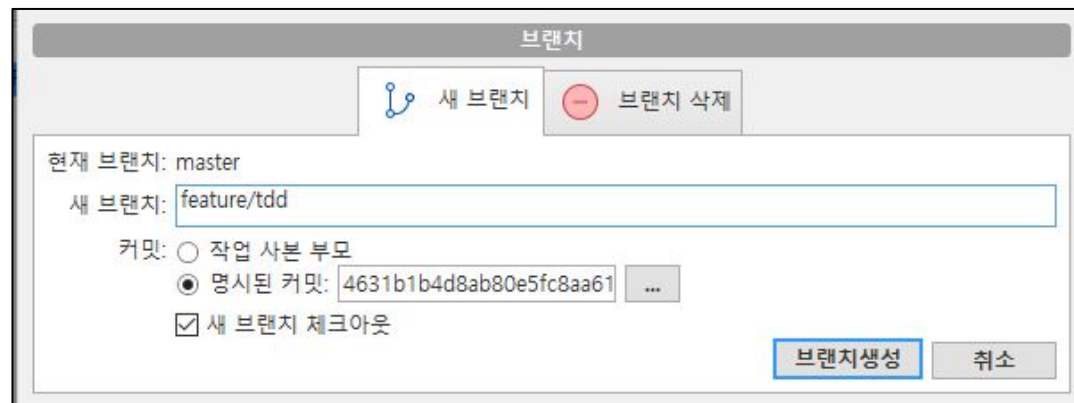
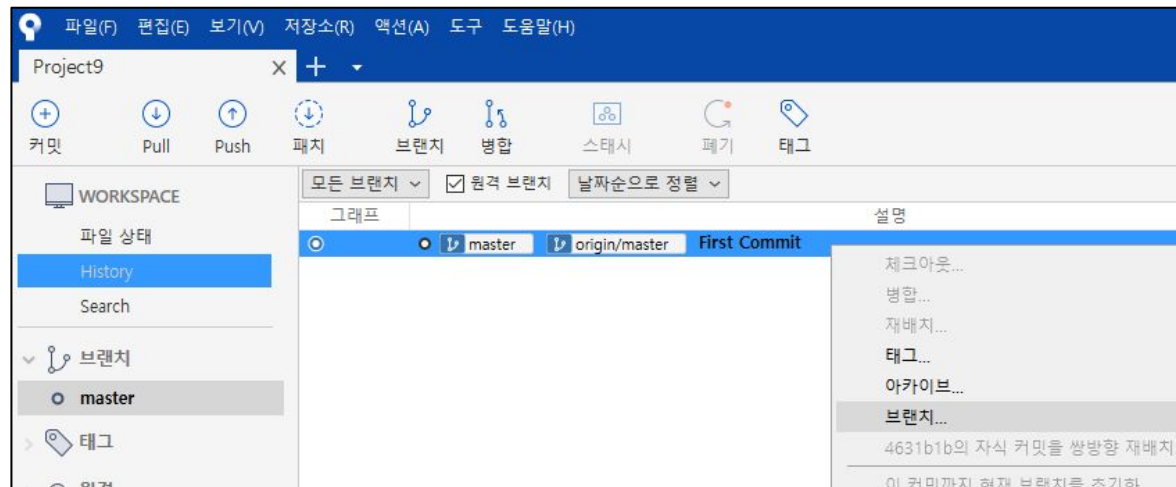
Push 진행 후 확인

✓ 정상적으로 Push 가 진행됨을 확인한다.



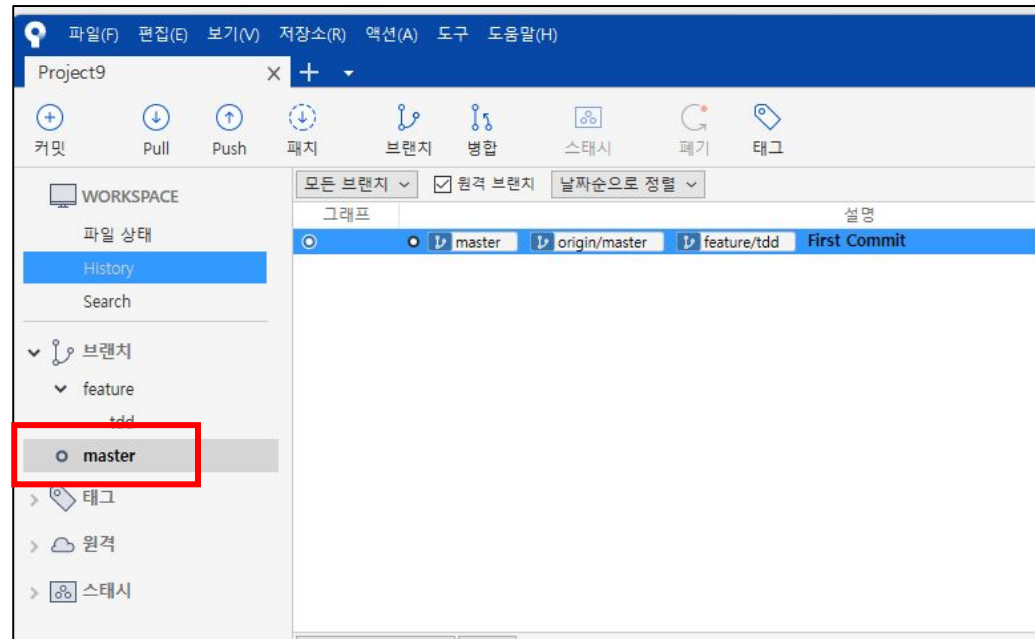
Branch 생성하기

✓ feature/tdd 브랜치를 추가한다.



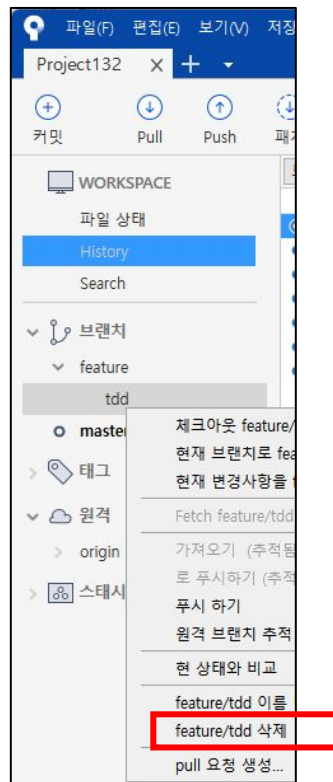
Branch 삭제하기 - 1

- ✓ Master branch 에서 삭제 해야한다.
따라서, master branch를 더블클릭하여 Checkout을 한다.



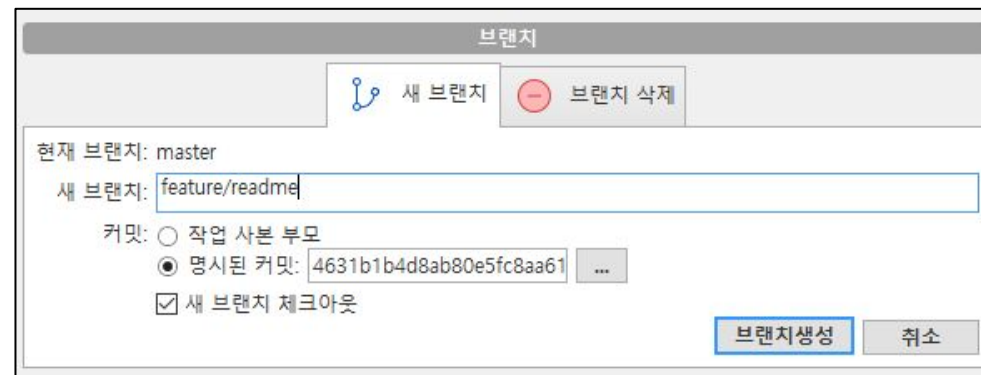
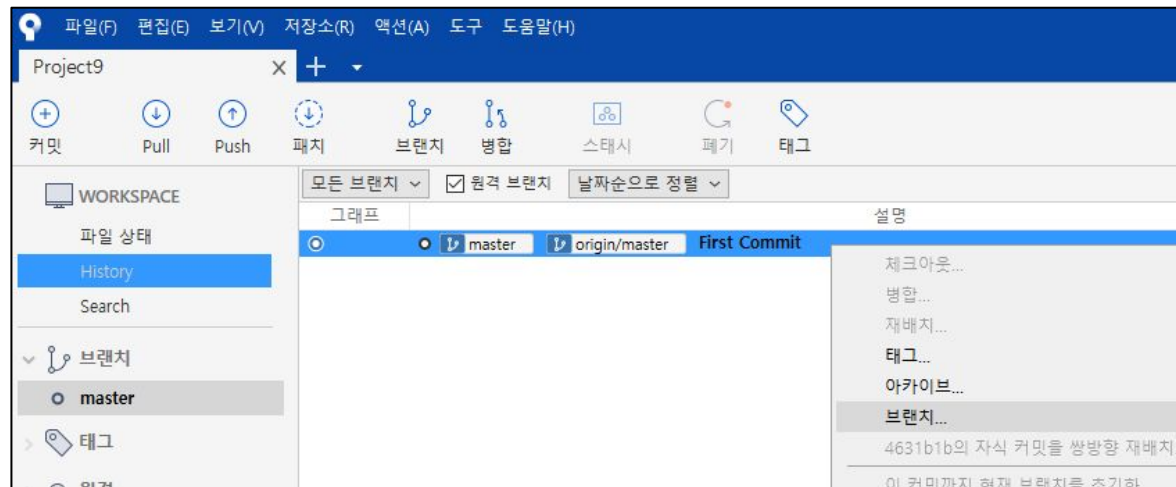
Branch 삭제하기 - 2

- ✓ 왼쪽 메뉴에서 feature/tdd 오른쪽 버튼을 누른 후 삭제 메뉴를 선택하여 삭제한다.



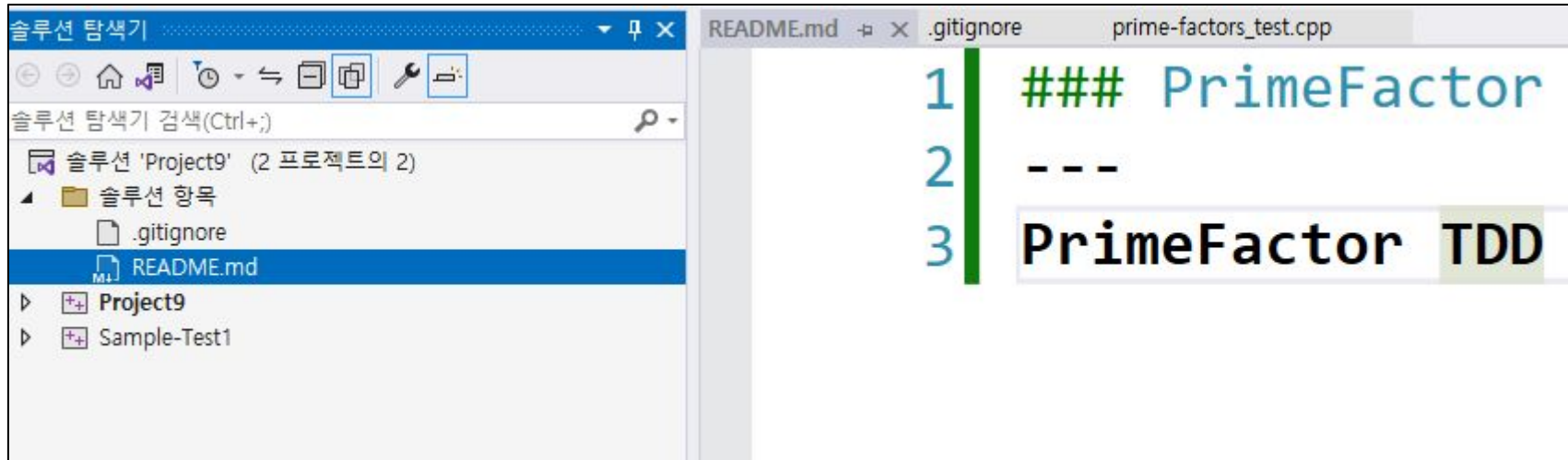
다시 Branch 생성하기

✓ feature/readme 브랜치를 추가한다.



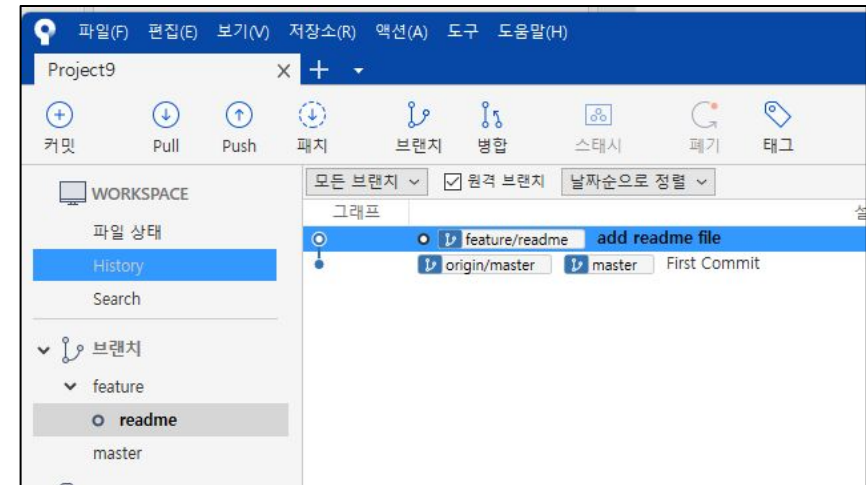
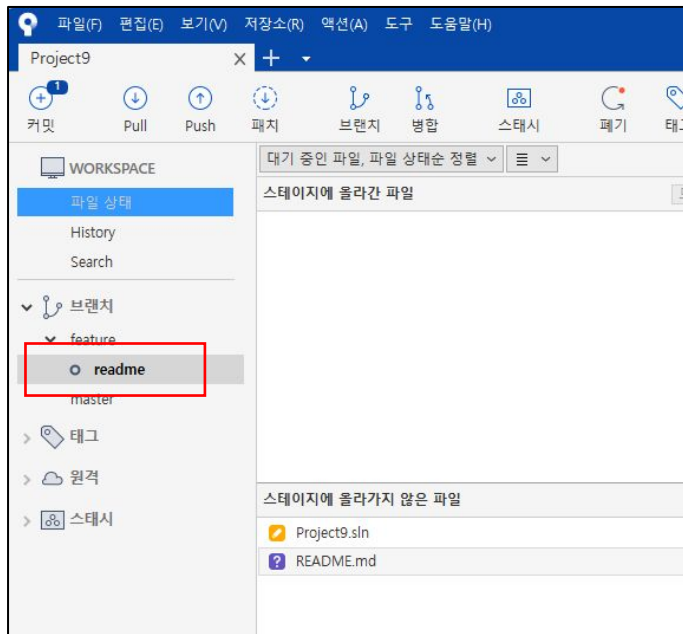
README.md 파일 추가

✓ 내용을 기입한다.



변경사항을 Commit한다.

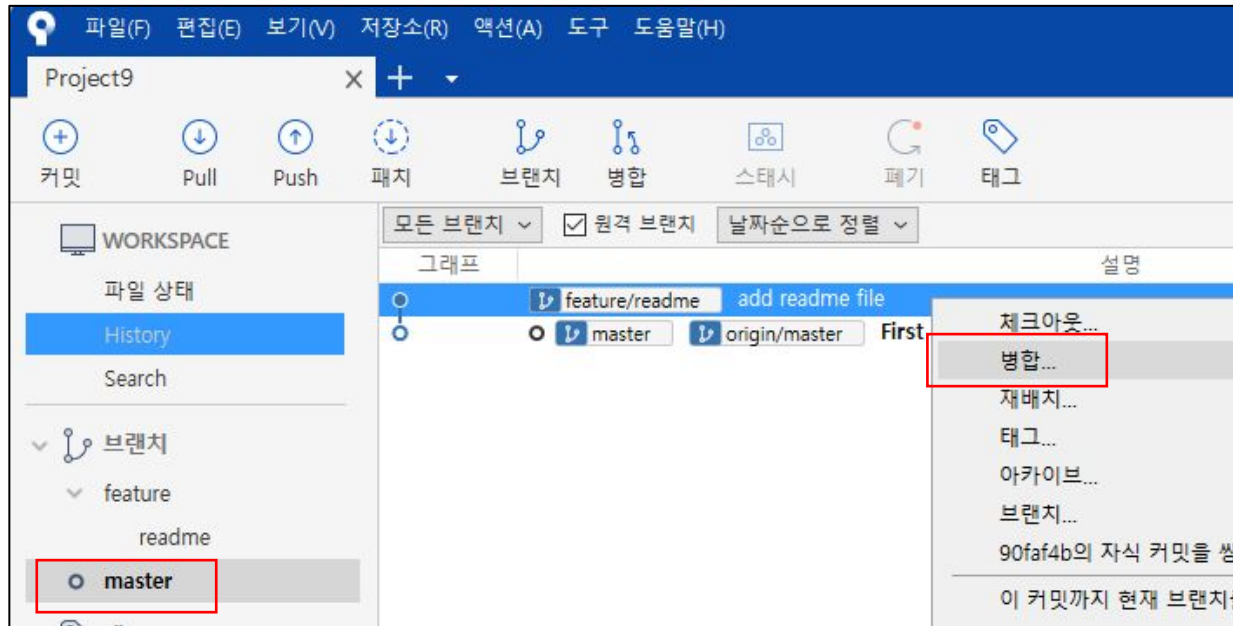
- ✓ master Branch가 아닌,
feature/readme 에 Commit 한다.



feature/readme 더블클릭 후
Commit 진행하기

Master Branch에서 병합하기

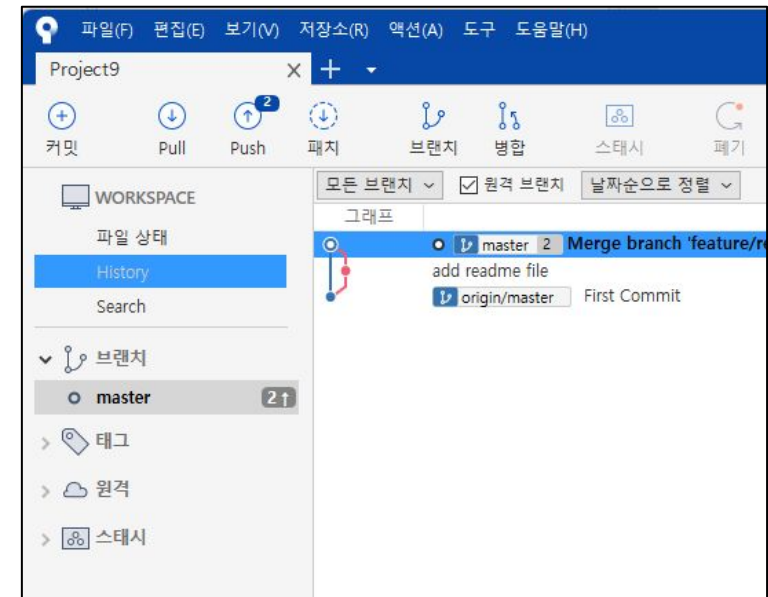
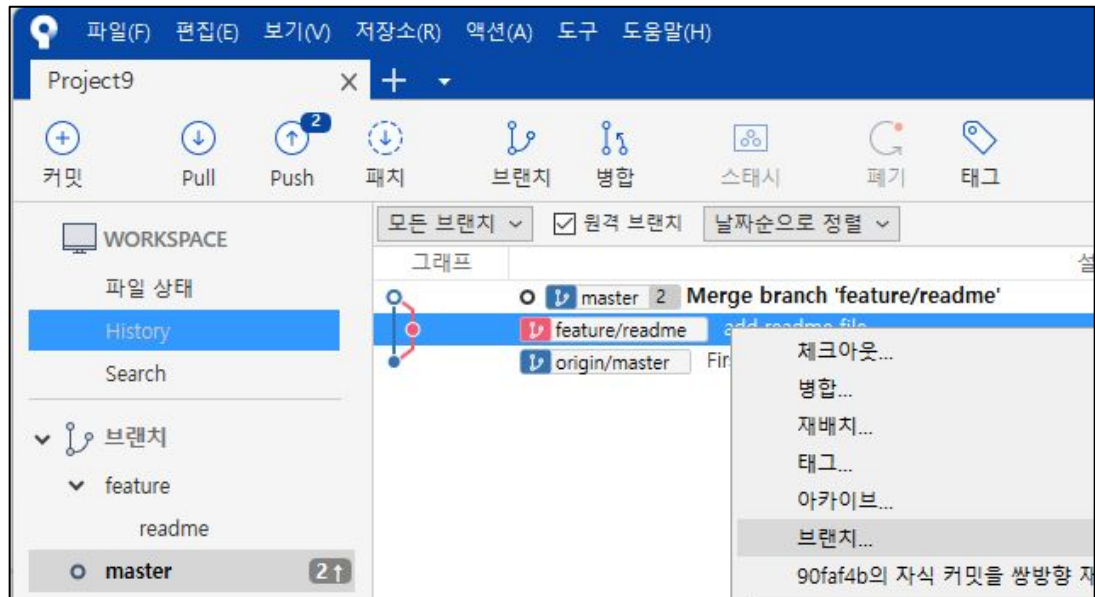
- ✓ master Branch로 체크아웃 한다.
- ✓ 그리고 Merge를 수행한다.



체크박스 필수!
Git 그래프를 보기 위해서
Merge 할때, 새로운 커밋을
추가하는 것으로 진행한다.

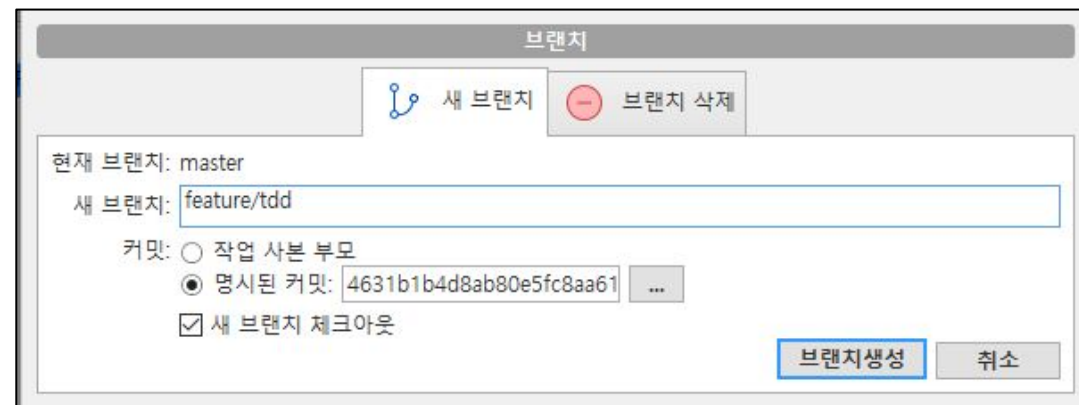
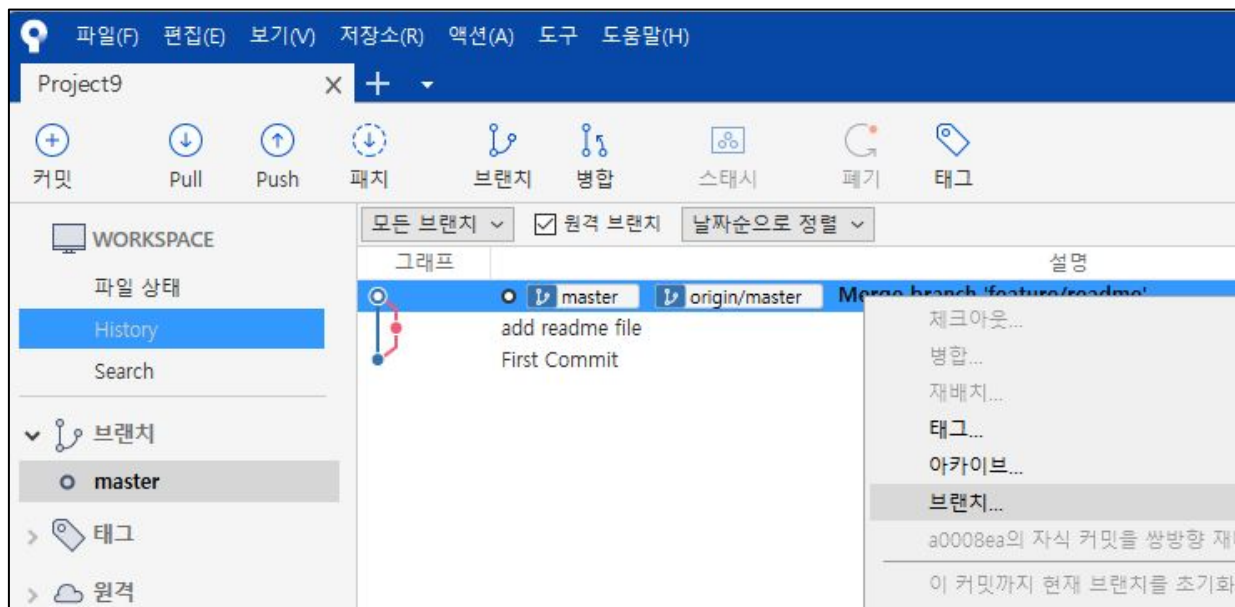
feature/readme 브랜치 삭제

- ✓ 해당 브랜치를 삭제한다.
- ✓ 완료된 후 Push 를 진행한다.



feature/tdd 브랜치 추가하기

✓ 다시 feature/tdd 브랜치를 추가한다.

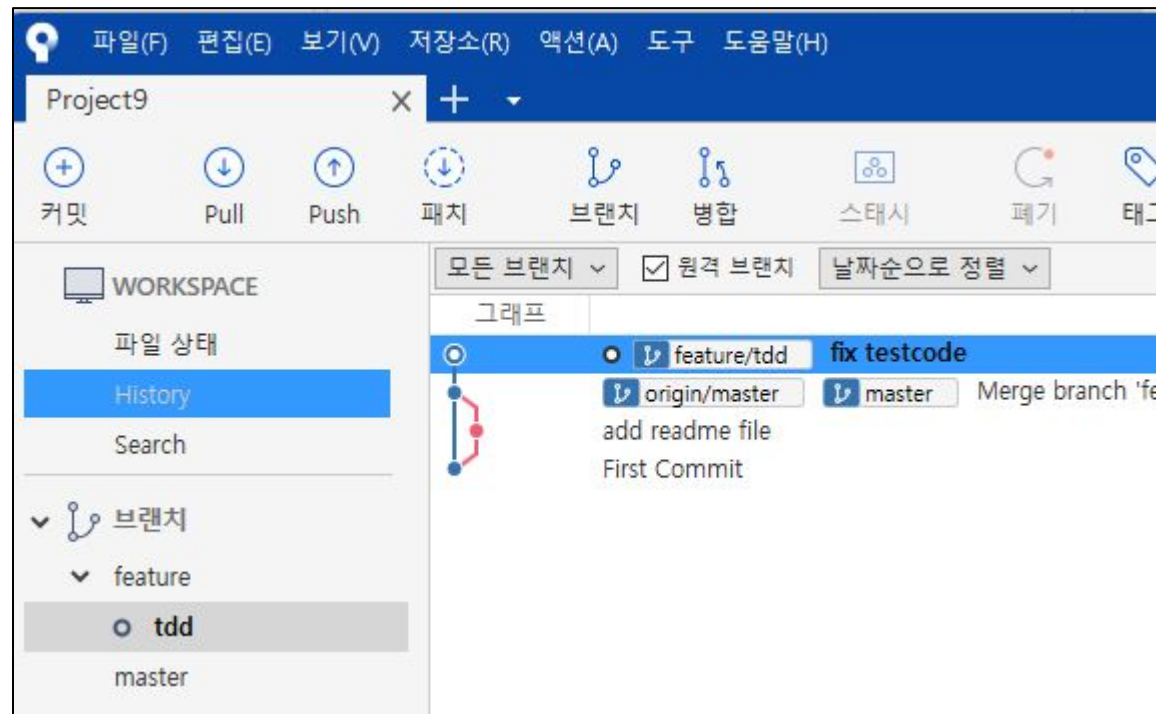


Commit 1회 진행

- ✓ master가 아닌,
feature/tdd 에 commit을 진행한다.

```
TEST(PrimeFactors, PrimTest) {  
    EXPECT_EQ(1, 1);  
    EXPECT_EQ(1, 1);  
    EXPECT_EQ(1, 1);  
    EXPECT_EQ(1, 1);  
}
```

테스트코드를 조금 수정한다.



Commit 진행

push 진행하기

- ✓ Push / Pull 의 단위는 Branch 이다.
- ✓ feature/tdd만 선택 후, Push 를 진행한다.

Push : Project9

다음 저장소에 푸시: origin https://github.com/mincoding1/PrimFactors-1423.git

푸시할 브랜치

푸시?	로컬 브랜치	리모트 브랜치	추적?
☒	feature/tdd	feature/tdd	☒
☐	master	master	☐

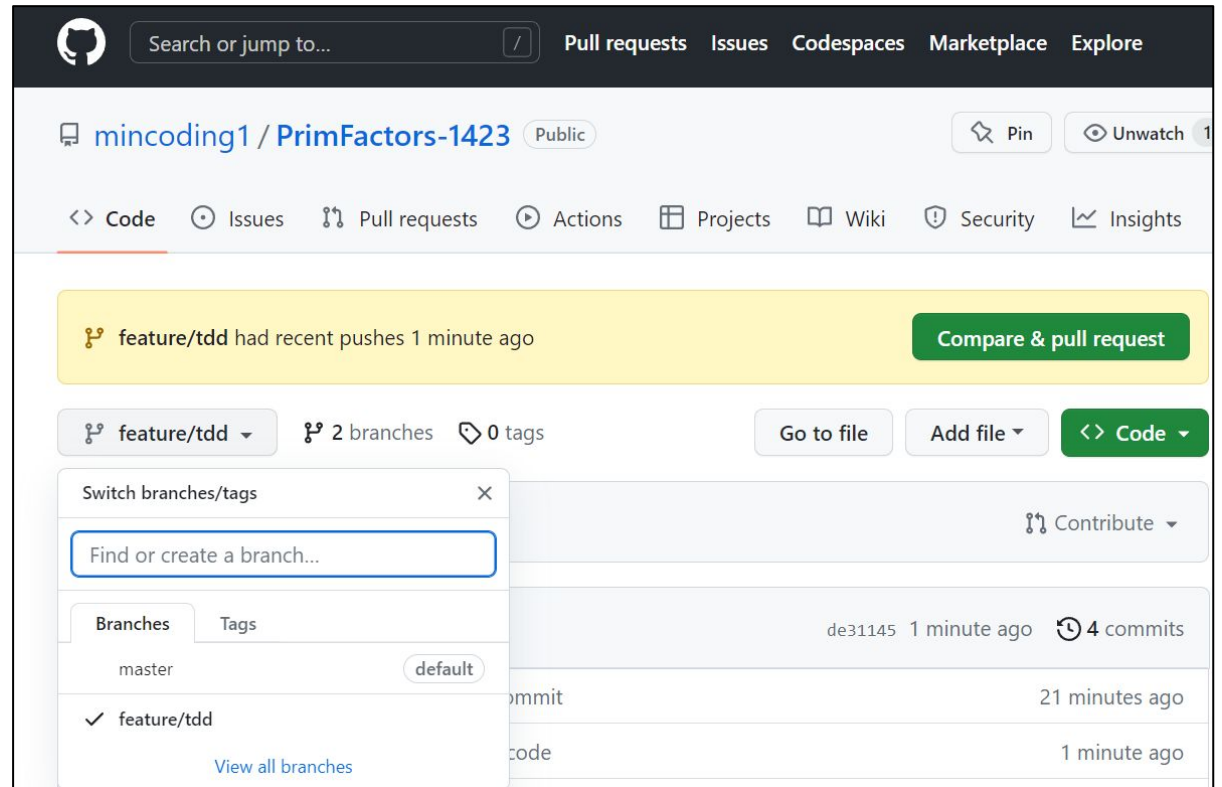
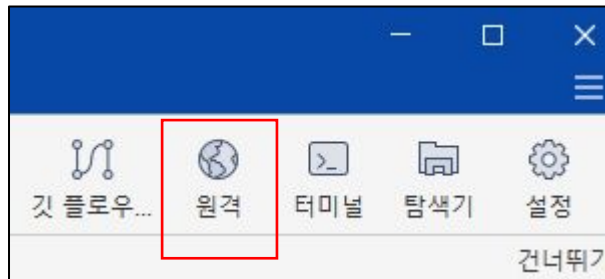
☒ 모두선택

☒ 모든 태그 푸시 ☐ 강제 푸시

Push 취소

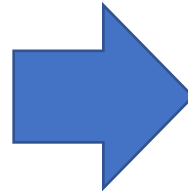
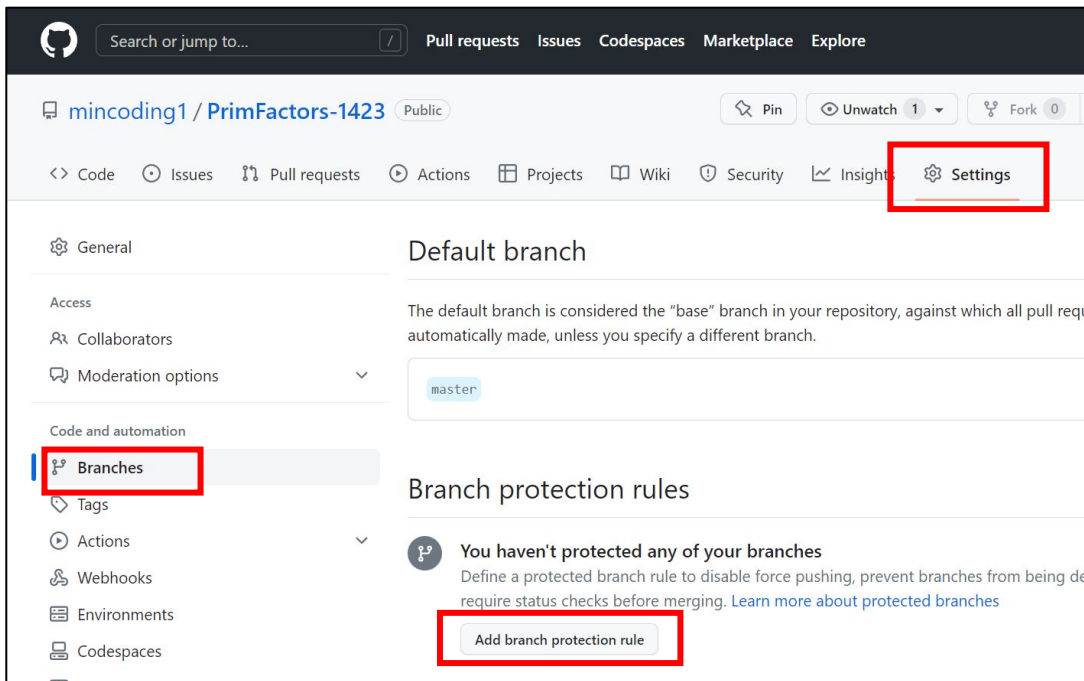
결과 확인

- ✓ Push 가 정상적으로 진행됨
- ✓ 이번 프로젝트에서는 해당 Branch 에서 개발 후, 최종적으로 Push한다. 그리고 PR진행한다.



Github 설정하기

- ✓ Branch Protection Rule
 - Approve 1인 이상 필수



Branch name pattern *

master

Protect matching branches

- ☒ **Require a pull request before merging**
When enabled, all commits must be made to a non-protected branch into a branch that matches this rule.
- ☒ **Require approvals**
When enabled, pull requests targeting a matching branch require a specified number of approvals before they can be merged.

Required number of approvals before merging: 1 ▼

Create

실습 과정 구성

- ✓ 본 실습 과정은 총 7개의 TDD Cycle Step 이 있다.
- ✓ 각 Step 은 다음과 같이 나누어져 있다.

1. Red
2. Green
3. Refactor

프로젝트 제출 방법

✓ Commit 타이밍

1. Red 단계에서는 Commit 하지 않는다.
2. Green 모두 수행 후, Unit Test Pass 시 **Commit** 을 한다.
3. Refactor 모두 수행 후, Unit Test 를 수행하며, Pass 시 **Commit**을 한다.

✓ Commit 메시지 헤더 Format

- Red 단계 : Commit 하지 않는다. (동작되는 코드만 Commit 해야 하기 때문)
- Green 단계 : **[feature] page 번호**
- Refactor 단계 : **[refactoring] page 번호**

✓ Push 하여 최종 제출하기

- 7 단계 까지 완료 이후, Push 1회만 진행

Prime Factors TDD 개발

Prime Factors 실습(01/07) : TDD Cycle 1

assert문을 완성

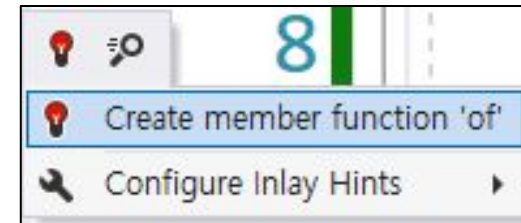
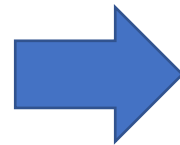
그리고 of 함수에 Alt + Enter를 눌러 메서드 자동생성

```
#include "pch.h"
#include "../Project9/prime-factors.cpp"

#include <vector>
using namespace std;

TEST(PrimeFactors, Of1) {
    PrimeFactor prime_factor;
    vector<int> expected = {};
    EXPECT_EQ(expected, prime_factor.of(1));
}
```

Test



of 라는 메서드를 자동생성하기 위해,
of 에 커서를 두고, Alt + Enter 누르기

of(1)의 소인수 분해는 불가능 하기에,
[] 빈 배열이 나오는 것을 기대함

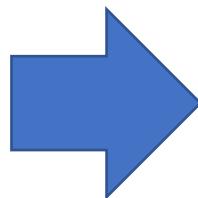
자동생성된 코드 수정

- ✓ Pass 될 정도로 구현한다.
 - Unit Test에 Pass가 되었으니, Commit 한다.

Production

```
class PrimeFactor {  
public:  
    void of(int i);  
};
```

탭키를 3회 누른다.



```
#include <vector>  
using namespace std;  
  
class PrimeFactor {  
public:  
    vector<int> of(int i) {  
        return {};  
    }  
};
```

✓ PrimeFactors (1 test) Success
✓ Of1 Success

Refactor 단계

임시 객체 대신,
result 객체를 만들어 return 한다.

- Unit Test에 Pass가 되었으니, Commit 한다.

```
class PrimeFactor {  
public:  
    std::vector<int> of(int i) {  
        vector<int> result = {};  
        return result;  
    }  
};
```

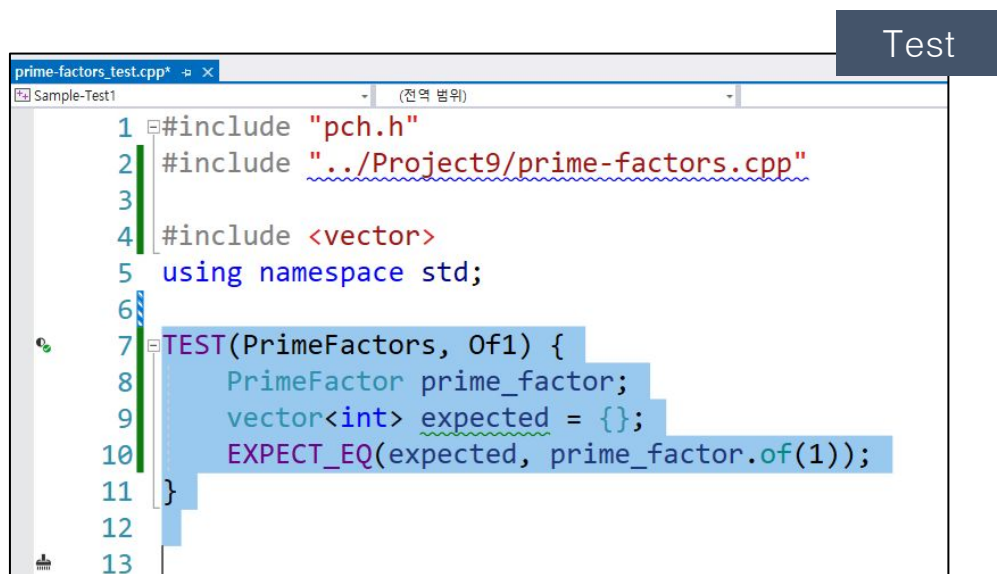
Production

result 객체를 추가한다.

▲ ✓ PrimeFactors (1 test) Success
✓ Of1 Success

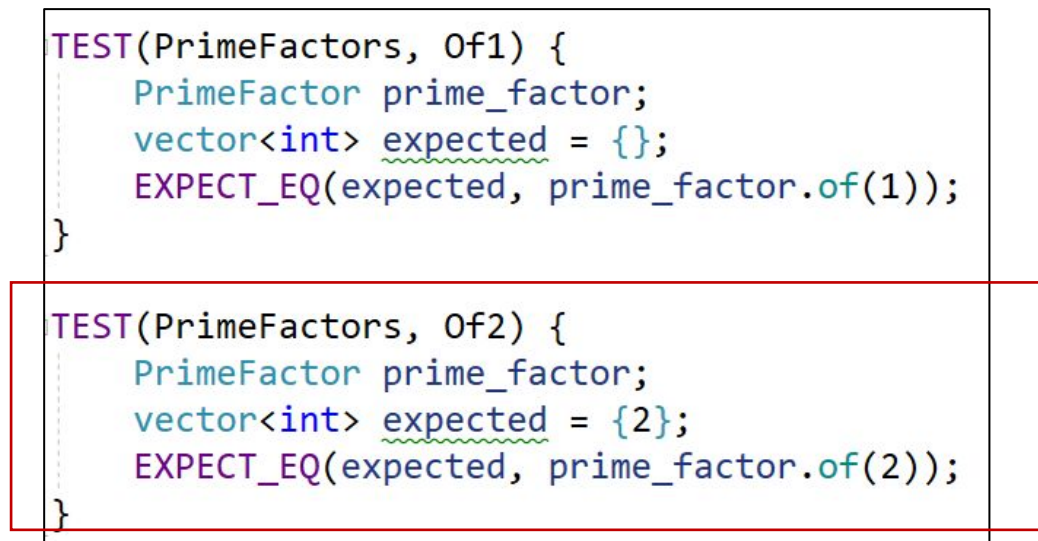
Prime Factors 실습(02/07) : TDD Cycle 2

✓ Red - '2'를 소인수분해 하는 테스트케이스 작성



```
1 #include "pch.h"
2 #include "../Project9/prime-factors.cpp"
3
4 #include <vector>
5 using namespace std;
6
7 TEST(PrimeFactors, Of1) {
8     PrimeFactor prime_factor;
9     vector<int> expected = {};
10    EXPECT_EQ(expected, prime_factor.of(1));
11 }
12
13
```

블록을 잡고, Ctrl + D (복제)



```
TEST(PrimeFactors, Of1) {
    PrimeFactor prime_factor;
    vector<int> expected = {};
    EXPECT_EQ(expected, prime_factor.of(1));
}

TEST(PrimeFactors, Of2) {
    PrimeFactor prime_factor;
    vector<int> expected = {2};
    EXPECT_EQ(expected, prime_factor.of(2));
}
```

코드 입력 후 Red를 눈으로 확인

▲	❌ PrimeFactors (2 tests) Failed: 1 test failed
✓	Of1 Success
❌	Of2 Failed

Fail 확인하기

✓ 기대 값과 달라 Fail을 눈으로 확인하기

- **[유의사항] Fail 되는 소스코드를 git에 commit 하면 안된다!**
- 이제 Pass 될 정도로만 구현을 시작하자.

```
Of2 failed

C:\Users\inho\source\repos\Project9\Sample-Test1\prime-factors_test.cpp(16):
error: Expected equality of these values:
  expected
    Which is: { 2 }
  prime_factor.of(2)
    Which is: {}
```

결과가 [2] 가 나와야 하는데,
실제 값은 빈 배열이 나왔으므로 Fail

Green 단계 시작

- ✓ 조금 Step을 높혀, Pass가 될 정도로만 구현하기
 - Unit Test에 Pass가 되었으니, Commit 한다.

Production

```
class PrimeFactor {
public:
    std::vector<int> of(int i) {
        vector<int> result = {};
        if (i == 2) {
            result.push_back(_val: 2);
        }
        return result;
    }
};
```

Pass가 될 정도로만,
코드 추가하기



```
class PrimeFactor {
public:
    std::vector<int> of(int number) {
        vector<int> result = {};
        if (number == 2) {
            result.push_back(_val: 2);
        }
        return result;
    }
};
```

변수명을 number로 변경
Baby Step 보다 더 큰 보폭으로,
여기 까지를 한 Step으로 진행함

▲ ✓ PrimeFactors (2 tests) Success
✓ Of1 Success
✓ Of2 Success

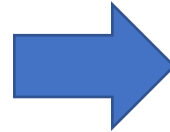
리팩토링 단계 - 중복 제거하기

- ✓ Test Code에서 중복을 제거하자.
 - Unit Test에 Pass가 되었으니, Commit 한다.

✓ PrimeFixture (2 tests)	Success
✓ Of1	Success
✓ Of2	Success

```
TEST(PrimeFactors, Of1) {  
    PrimeFactor prime_factor;  
    vector<int> expected = {};  
    EXPECT_EQ(expected, prime_factor.of(1));  
}  
  
TEST(PrimeFactors, Of2) {  
    PrimeFactor prime_factor;  
    vector<int> expected = {2};  
    EXPECT_EQ(expected, prime_factor.of(2));  
}
```

Test



```
class PrimeFixture : public testing::Test {  
public:  
    PrimeFactor prime_factor;  
    vector<int> expected;  
};  
  
TEST_F(PrimeFixture, Of1) {  
    expected = {};  
    EXPECT_EQ(expected, prime_factor.of(1));  
}  
  
TEST_F(PrimeFixture, Of2) {  
    expected = {2};  
    EXPECT_EQ(expected, prime_factor.of(2));  
}
```

Test Fixture 를 만들고,
모든 TEST 메서드를 TEST_F로 변경

Prime Factors 실습(03/07) : TDD Cycle 3

✓ Red - '3'을 소인수분해 하는 테스트케이스 작성

```
TEST_F(PrimeFixture, Of1) {  
    expected = {};  
    EXPECT_EQ(expected, prime_factor.of(1));  
}  
  
TEST_F(PrimeFixture, Of2) {  
    expected = {2};  
    EXPECT_EQ(expected, prime_factor.of(2));  
}  
  
TEST_F(PrimeFixture, Of3) {  
    expected = {3};  
    EXPECT_EQ(expected, prime_factor.of(3));  
}
```

Test

▲	PrimeFixture (3 tests)	Failed: 1 test fail
✓	Of1	Success
✓	Of2	Success
✗	Of3	Failed

위 코드를 복사 붙여넣기 진행.
(Ctrl + D)

테스트 내용 : 소인수 3을 넣었을 때 [3] 이 나와야 함.

Test에 성공할 정도로만 구현하기

- ✓ Green – 테스트케이스 성공
 - Unit Test에 Pass가 되었으니, Commit 한다.

Production

```
class PrimeFactor {  
public:  
    std::vector<int> of(int number) {  
        vector<int> result = {};  
        if (number == 2) {  
            result.push_back(_val: 2);  
        }  
        if (number == 3) {  
            result.push_back(_val: 3);  
        }  
        return result;  
    }  
};
```

코드를 추가하고,
Green을 눈으로 확인

▲ ✓ PrimeFixture (3 tests)	Success
✓ Of1	Success
✓ Of2	Success
✓ Of3	Success

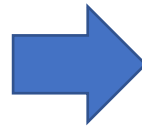
리팩토링 - 테스트 코드

✓ Production Code, Refactoring 수행

```
class PrimeFactor {  
public:  
    std::vector<int> of(int number) {  
        vector<int> result = {};  
        if (number == 2) {  
            result.push_back(_Val: 2);  
        }  
        if (number == 3) {  
            result.push_back(_Val: 3);  
        }  
        return result;  
    }  
};
```

Production

하드코딩 코드를 number 변수로 변경



```
class PrimeFactor {  
public:  
    std::vector<int> of(int number) {  
        vector<int> result = {};  
        if (number == 2) {  
            result.push_back(number);  
        }  
        if (number == 3) {  
            result.push_back(number);  
        }  
        return result;  
    }  
};
```

리팩토링 후 Test로 결과 확인

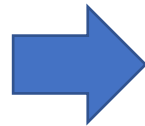
▲ ✓ PrimeFixture (3 tests)	Success
✓ Of1	Success
✓ Of2	Success
✓ Of3	Success

리팩토링 - 더 Generalize 하도록 개선

- ✓ Refactoring – generalize, 더 범용적인 코드로 리팩토링 함
 - Unit Test에 Pass가 되었으니, Commit 한다.

Production

```
class PrimeFactor {
public:
    std::vector<int> of(int number) {
        vector<int> result = {};
        if (number == 2) {
            result.push_back(number);
        }
        if (number == 3) {
            result.push_back(number);
        }
        return result;
    }
};
```



```
class PrimeFactor {
public:
    std::vector<int> of(int number) {
        vector<int> result = {};
        if (number > 1) {
            result.push_back(number);
        }
        return result;
    }
};
```

▲ ✓ PrimeFixture (3 tests)	Success
✓ Of1	Success
✓ Of2	Success
✓ Of3	Success

Prime Factors 실습(04/07) : TDD Cycle 4

✓ Red – ‘4’를 소인수분해 하는 테스트케이스 작성

```
TEST_F(PrimeFixture, Of4) {  
    expected = {2, 2};  
    EXPECT_EQ(expected, prime_factor.of(4));  
}
```

Test

4를 소인수 분해하면, [2, 2] 가 나와야하는
테스트 코드 추가하기

PrimeFixture (4 tests)		Failed: 1 test failed
✓ Of1	Success	
✓ Of2	Success	
✓ Of3	Success	
✗ Of4	Failed	

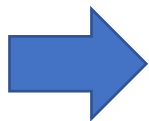
4일 때, 소인수 분해가 되도록 코드 추가

✓ Pass만 되도록 구현

- Unit Test에 Pass가 되었으니, Commit 한다.

Production

```
class PrimeFactor {
public:
    std::vector<int> of(int number) {
        vector<int> result = {};
        if (number > 1) {
            result.push_back(number);
        }
        return result;
    }
};
```



```
class PrimeFactor {
public:
    std::vector<int> of(int number) {
        vector<int> result = {};
        if (number > 1) {
            if (number == 4) {
                result.push_back(_Val: 2);
                result.push_back(_Val: 2);
            }
            else {
                result.push_back(number);
            }
        }
        return result;
    }
};
```

✓ PrimeFixture (4 tests)	Success
✓ Of1	Success
✓ Of2	Success
✓ Of3	Success
✓ Of4	Success

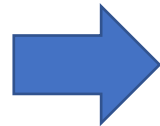
4인 경우, [2, 2]가 되고,
그 외는 기존처럼 [number]가 되도록 함

리팩토링 - 더 Generalize하도록 코드 수정

✓ Refactoring - generalize

Production

```
class PrimeFactor {
public:
    std::vector<int> of(int number) {
        vector<int> result = {};
        if (number > 1) {
            if (number == 4) {
                result.push_back(_Val: 2);
                result.push_back(_Val: 2);
            }
            else {
                result.push_back(number);
            }
        }
        return result;
    }
};
```



조금 더 범용적이도록
소스코드 리팩토링

```
class PrimeFactor {
public:
    std::vector<int> of(int number) {
        vector<int> result = {};
        if (number > 1) {
            if (number == 4) {
                if (number % 2 == 0) {
                    result.push_back(_Val: 2);
                    number /= 2;
                }

                if (number % 2 == 0) {
                    result.push_back(_Val: 2);
                    number /= 2;
                }
            }
            else {
                result.push_back(number);
            }
        }
        return result;
    }
};
```

✓ PrimeFixture (4 tests)	
✓ Of1	Success
✓ Of2	Success
✓ Of3	Success
✓ Of4	Success

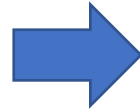
리팩토링 - if를 while로 변경

- ✓ Refactoring - if 반복구문 리팩토링
 - Unit Test에 Pass가 되었으니, Commit 한다.

Production

```
class PrimeFactor {
public:
    std::vector<int> of(int number) {
        vector<int> result = {};
        if (number > 1) {
            if (number == 4) {
                if (number % 2 == 0) {
                    result.push_back(_val: 2);
                    number /= 2;
                }

                if (number % 2 == 0) {
                    result.push_back(_val: 2);
                    number /= 2;
                }
            }
            else {
                result.push_back(number);
            }
        }
        return result;
    }
};
```



조금 더 범용적이도록
소스코드 리팩토링

```
class PrimeFactor {
public:
    std::vector<int> of(int number) {
        vector<int> result = {};
        if (number > 1) {
            if (number == 4) {
                while (number % 2 == 0) {
                    result.push_back(_val: 2);
                    number /= 2;
                }
            }
            else {
                result.push_back(number);
            }
        }
        return result;
    }
};
```

✓ PrimeFixture (4 tests)		Success
✓ Of1		Success
✓ Of2		Success
✓ Of3		Success
✓ Of4		Success

Prime Factors 실습(05/07) : TDD Cycle 5

✓ Red – ‘6’을 소인수분해 하는 테스트케이스 작성

```
TEST_F(PrimeFixture, Of6) {  
    expected = {2, 3};  
    EXPECT_EQ(expected, prime_factor.of(6));  
}
```

PrimeFixture (5 tests)		Failed: 1 test failed
✓ Of1		Success
✓ Of2		Success
✓ Of3		Success
✓ Of4		Success
✗ Of6		Failed

Green – 6의 소인수 분해 정답 나오도록 코드 추가

✓ Pass만 되도록 구현

- Unit Test에 Pass가 되었으니, Commit 한다.

Production

```
class PrimeFactor {
public:
    std::vector<int> of(int number) {
        vector<int> result = {};
        if (number > 1) {
            if (number == 4) {
                while (number % 2 == 0) {
                    result.push_back(_Val: 2);
                    number /= 2;
                }
            }
            else {
                result.push_back(number);
            }
        }
        return result;
    }
};
```



```
class PrimeFactor {
public:
    std::vector<int> of(int number) {
        vector<int> result = {};
        if (number > 1) {
            if (number == 4) {
                while (number % 2 == 0) {
                    result.push_back(_Val: 2);
                    number /= 2;
                }
            }
            else if (number == 6) {
                result.push_back(_Val: 2);
                result.push_back(_Val: 3);
            }
            else {
                result.push_back(number);
            }
        }
        return result;
    }
};
```

6일때 [2, 3]이 나오도록 코드 추가

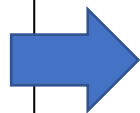
✓ PrimeFixture (5 tests)	Success
✓ Of1	Success
✓ Of2	Success
✓ Of3	Success
✓ Of4	Success
✓ Of6	Success

더 범용적으로 리팩토링

✓ Refactoring – generalize

Production

```
class PrimeFactor {
public:
    std::vector<int> of(int number) {
        vector<int> result = {};
        if (number > 1) {
            if (number == 4) {
                while (number % 2 == 0) {
                    result.push_back(_Val: 2);
                    number /= 2;
                }
            }
            else if (number == 6) {
                result.push_back(_Val: 2);
                result.push_back(_Val: 3);
            }
            else {
                result.push_back(number);
            }
        }
        return result;
    }
};
```



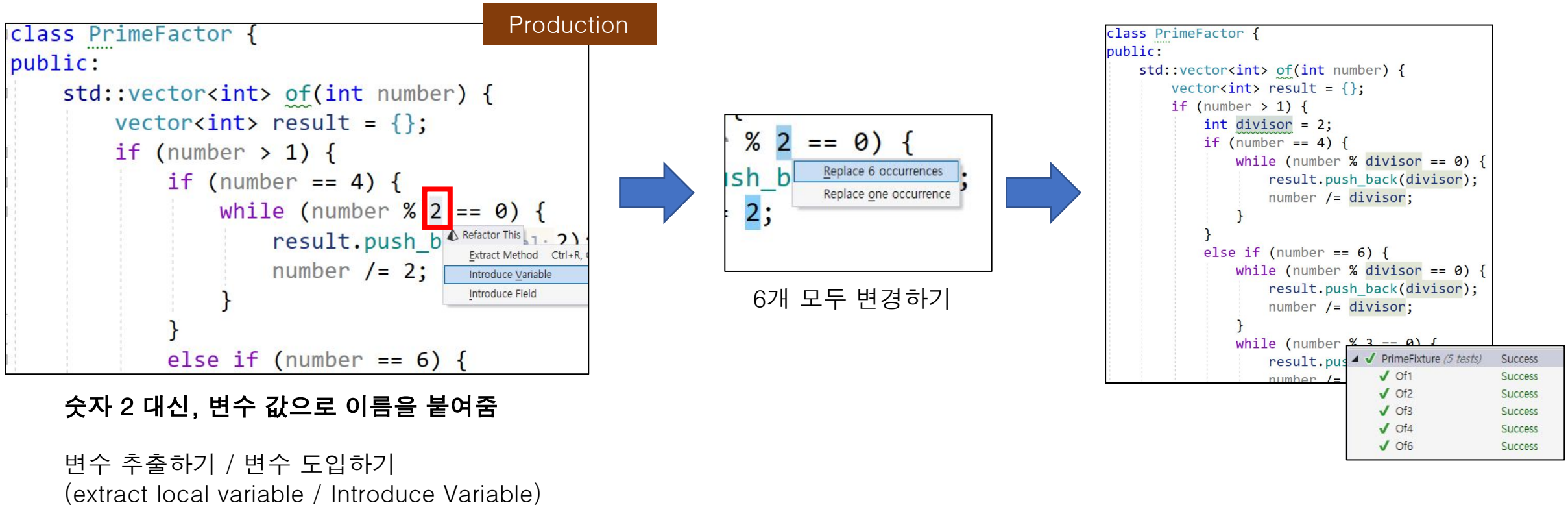
조금 더 범용적이도록
소스코드 리팩토링

```
class PrimeFactor {
public:
    std::vector<int> of(int number) {
        vector<int> result = {};
        if (number > 1) {
            if (number == 4) {
                while (number % 2 == 0) {
                    result.push_back(_Val: 2);
                    number /= 2;
                }
            }
            else if (number == 6) {
                while (number % 2 == 0) {
                    result.push_back(_Val: 2);
                    number /= 2;
                }
                while (number % 3 == 0) {
                    result.push_back(_Val: 3);
                    number /= 3;
                }
            }
            else {
                result.push_back(number);
            }
        }
        return result;
    }
};
```

PrimeFixture (5 tests)	Success
✓ Of1	Success
✓ Of2	Success
✓ Of3	Success
✓ Of4	Success
✓ Of6	Success

리팩토링 – Extract local variable (변수 추출하기)

✓ Refactoring – extract local variable



리팩토링 - divisor로 변경

✓ 숫자 3 대신, divisor 값으로 변경

Production

```
else if (number == 6) {  
    while (number % divisor == 0) {  
        result.push_back(divisor);  
        number /= divisor;  
    }  
    while (number % 3 == 0) {  
        result.push_back(_val: 3);  
        number /= 3;  
    }  
}  
else {  
    result.push_back(number);  
}
```



```
else if (number == 6) {  
    while (number % divisor == 0) {  
        result.push_back(divisor);  
        number /= divisor;  
    }  
    divisor++;  
    while (number % divisor == 0) {  
        result.push_back(divisor);  
        number /= divisor;  
    }  
    divisor++;  
}  
else {  
    result.push_back(number);  
}
```

divisor++ 한 값으로 코드 수정

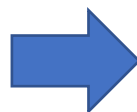
✓ PrimeFixture (5 tests)		Success
✓ Of1		Success
✓ Of2		Success
✓ Of3		Success
✓ Of4		Success
✓ Of6		Success

리팩토링 - for loop로 변경

- ✓ Refactoring - 비슷한 중복을 완전한 중복으로 변환
 - Unit Test에 Pass가 되었으니, Commit 한다.

Production

```
else if (number == 6) {  
    while (number % divisor == 0) {  
        result.push_back(divisor);  
        number /= divisor;  
    }  
    divisor++;  
    while (number % divisor == 0) {  
        result.push_back(divisor);  
        number /= divisor;  
    }  
    divisor++;  
}  
else {  
    result.push_back(number);  
}
```



```
else if (number == 6) {  
    for (divisor = 2; number > 1; divisor++) {  
        while (number % divisor == 0) {  
            result.push_back(divisor);  
            number /= divisor;  
        }  
    }  
}  
else {  
    result.push_back(number);  
}
```

for문 divisor / number 순서 유의하여
코드 작성하기

✓ PrimeFixture (5 tests)		Success
✓ Of1		Success
✓ Of2		Success
✓ Of3		Success
✓ Of4		Success
✓ Of6		Success

Prime Factors 실습(06/07) : TDD Cycle 6

✓ Red – ‘9’를 소인수분해 하는 테스트케이스 작성

```
TEST_F(PrimeFixture, Of9) {  
    expected = {3, 3};  
    EXPECT_EQ(expected, prime_factor.of(9));  
}
```

새로운 테스트코드 추가
소인수분해 9일때 정답은 [3, 3]이어야함

PrimeFixture (6 tests)		Failed: 1 test failed
✓	Of1	Success
✓	Of2	Success
✓	Of3	Success
✓	Of4	Success
✓	Of6	Success
✗	Of9	Failed

Green – 하드코딩으로 Green 만들기

✓ Pass만 되도록 구현

- Unit Test에 Pass가 되었으니, Commit 한다.

```
else if (number == 6) {  
    for (divisor = 2; number > 1; divisor++) {  
        while (number % divisor == 0) {  
            result.push_back(divisor);  
            number /= divisor;  
        }  
    }  
}  
  
else if (number == 9) {  
    result.push_back(_Val: 3);  
    result.push_back(_Val: 3);  
}  
  
else {  
    result.push_back(number);  
}
```

Production

PrimeFixture (6 tests)		Success
✓ Of1		Success
✓ Of2		Success
✓ Of3		Success
✓ Of4		Success
✓ Of6		Success
✓ Of9		Success

number == 9 일때 코드 추가

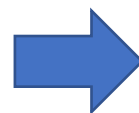
리팩토링 - 중복 코드 제거하기

✓ Refactoring - 중복제거

```
else if (number == 6) {  
    for (divisor = 2; number > 1; divisor++) {  
        while (number % divisor == 0) {  
            result.push_back(divisor);  
            number /= divisor;  
        }  
    }  
}  
  
else if (number == 9) {  
    result.push_back(_Val: 3);  
    result.push_back(_Val: 3);  
}  
  
else {  
    result.push_back(number);  
}
```

Production

사각 영역, 소스코드 삭제



```
else if (number == 6 || number == 9) {  
    for (divisor = 2; number > 1; divisor++) {  
        while (number % divisor == 0) {  
            factors.add(divisor);  
            number /= divisor;  
        }  
    }  
}  
  
else {  
    factors.add(number);  
}
```

코드 추가하기

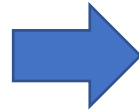
✓ PrimeFixture (6 tests)		Success
✓ Of1		Success
✓ Of2		Success
✓ Of3		Success
✓ Of4		Success
✓ Of6		Success
✓ Of9		Success

리팩토링 - 중복 코드 제거하기

- ✓ Refactoring - 중복제거
 - Unit Test에 Pass가 되었으니, Commit 한다.

Production

```
class PrimeFactor {
public:
    std::vector<int> of(int number) {
        vector<int> result = {};
        if (number > 1) {
            int divisor = 2;
            if (number == 4) {
                while (number % divisor == 0) {
                    result.push_back(divisor);
                    number /= divisor;
                }
            }
            else if (number == 6 || number == 9) {
                for (divisor = 2; number > 1; divisor++) {
                    while (number % divisor == 0) {
                        result.push_back(divisor);
                        number /= divisor;
                    }
                }
            }
            else {
                result.push_back(number);
            }
        }
        return result;
    }
};
```



```
class PrimeFactor {
public:
    std::vector<int> of(int number) {
        vector<int> result = {};
        if (number > 1) {
            int divisor = 2;
            if (number == 4 || number == 6 || number == 9) {
                for (divisor = 2; number > 1; divisor++) {
                    while (number % divisor == 0) {
                        result.push_back(divisor);
                        number /= divisor;
                    }
                }
            }
            else {
                result.push_back(number);
            }
        }
        return result;
    }
};
```

✓ PrimeFixture (6 tests)		Success
✓ Of1		Success
✓ Of2		Success
✓ Of3		Success
✓ Of4		Success
✓ Of6		Success
✓ Of9		Success

else if를 if로 변경하고,
number == 4 || 내용 추가.

사각 영역, 소스코드 삭제

Prime Factors 실습(07/07) : TDD Cycle 7

✓ Red – '12'를 소인수분해 하는 테스트케이스 작성

```
TEST_F(PrimeFixture, Of12) {  
    expected = {2, 2, 3};  
    EXPECT_EQ(expected, prime_factor.of(12));  
}
```

Test

12 소인수분해시
[2, 2, 3]이 나와야 한다는 테스트코드 추가.

PrimeFixture (7 tests)		Failed: 1 test failed
✓ Of1		Success
✗ Of12		Failed
✓ Of2		Success
✓ Of3		Success
✓ Of4		Success
✓ Of6		Success
✓ Of9		Success

Green - 12일때 소스코드 추가.

✓ Pass만 되도록 구현

- Unit Test에 Pass가 되었으니, Commit 한다.

```
class PrimeFactor {  
public:  
    std::vector<int> of(int number) {  
        vector<int> result = {};  
        if (number > 1) {  
            int divisor = 2;  
            if (number == 4 || number == 6 || number == 9 || number == 12) {  
                for (divisor = 2; number > 1; divisor++) {  
                    while (number % divisor == 0) {  
                        result.push_back(divisor);  
                        number /= divisor;  
                    }  
                }  
            }  
            else {  
                result.push_back(number);  
            }  
        }  
        return result;  
    }  
};
```

Production

✓ PrimeFixture (7 tests)	Success
✓ Of1	Success
✓ Of12	Success
✓ Of2	Success
✓ Of3	Success
✓ Of4	Success
✓ Of6	Success
✓ Of9	Success

number == 12 일때 코드 추가 후 Green 확인하기

리팩토링

- ✓ Refactoring – 중복코드 제거, generalize
 - Unit Test에 Pass가 되었으니, Commit 한다.

```
class PrimeFactor {
public:
    std::vector<int> of(int number) {
        vector<int> result = {};
        if (number > 1) {
            int divisor = 2;
            if (number == 4 || number == 6 || number == 9 || number == 12) {
                for (divisor = 2; number > 1; divisor++) {
                    while (number % divisor == 0) {
                        result.push_back(divisor);
                        number /= divisor;
                    }
                }
            }
            else {
                result.push_back(number);
            }
        }
        return result;
    }
};
```

Production



```
class PrimeFactor {
public:
    std::vector<int> of(int number) {
        vector<int> result = {};

        for (int divisor = 2; number > 1; divisor++) {
            while (number % divisor == 0) {
                result.push_back(divisor);
                number /= divisor;
            }
        }

        return result;
    }
};
```

for (int divisor = 2 ..) 로 수정

PrimeFixture (7 tests)		Success
✓ Of1		Success
✓ Of12		Success
✓ Of2		Success
✓ Of3		Success
✓ Of4		Success
✓ Of6		Success
✓ Of9		Success

블록 영역, 소스코드 제거